Machine Learning 2 (APM_50441_EP/DSAIB - 2026-2027). Lab7

The objectives of the lab

This lab illustrates the seventh lecture (*At Scale*) on the flights of the United States in 2015. Its goals are:

- to read and store a large dataset efficiently (binary columnar formats, Parquet, partitions), and to use a sample when it is enough,
- to compare engines for the split/apply/combine computations on the full data, in memory (`dplyr`, `data.table`, `duckdb`) and directly on files (`arrow`, `duckdb`),
- to profile and optimize code, from the algorithm to the compiled language (`profvis`, `bench`, `Rcpp`),
- to parallelize a cross validation, and to measure when it pays off (`mirai`, `purrr::in_parallel()`),
- to serve a model with a REST API (`plumber2`) and a web application (`shiny` and `bslib`).

The optional *To go further* part, after the core, can be skipped at first: it covers Map/Reduce by hand, Spark, larger-than-memory computations, the memory hierarchy, pipelines, in-database scoring and the deployment of models.

> **Use of AI.** AI assistants (e.g. the ones built into Colab or VS Code) are allowed, but you must be able to explain every line you use, and answer the *predict before you run* questions before running the code.

Core (about 2 hours)

The full dataset (5.8 million flights, 1.4 GB in memory) is too large for the computations of Parts 1, 3, 4 and 5 on some laptops: they use a random sample of 10% of the flights (582 000 flights). Part 2 uses the full data, stored as Parquet files (138 MB, downloaded once): it needs about 4 GB of memory, which Colab and most laptops have.

Exercise 1 (**Reading, storing and sampling**) 1. *Predict before you run:* which reader is the fastest? Read `data/flights_sample.csv.gz` with `readr::read_csv`, `data.table::fread` and `arrow::read_csv_arrow`, and compare their times with `bench::mark` (3 iterations, `check = FALSE`). What is the size of the data in memory (`lobstr::obj_size`)?

2. Write the flights as a Parquet dataset partitioned by `MONTH` (`arrow::write_dataset` in `data/flights_parquet`), and compare the size on disk with the CSV file. Then open it with `open_dataset`, and compute the mean arrival delay of the flights of December for each airline *without* loading the data in memory (`filter`, `group_by`, `summarise`, then `collect`). Compare the time with the same computation from the CSV file.

   The *sampling trick* (lecture): for many questions, a random sample is enough, as the precision of an estimate depends on the size of the sample. Often, we do not need the full population to obtain a good enough estimate (PC7, *Reservoir sampling*).

3. Estimate the mean arrival delay of all the flights of the sample with random subsamples of $n = 100$, 1000, 10 000 and 100 000 flights (200 subsamples each): compare the standard deviation of the estimates with $\sigma/\sqrt{n}$. Then compare a simple random subsample of 2004 flights with a subsample *stratified* by month (167 flights each month, the monthly means being weighted by the proportions of the months).

Exercise 2 (**Split, apply, combine**)

On the 10% sample, every engine computes such aggregates in a few tens of milliseconds: the fixed costs (parsing the query, starting the threads, converting the results) dominate and hide the differences between the engines. This part uses the full data: 5.8 million flights, stored as 12 Parquet files (one per month, 138 MB in all) downloaded once into `data/flights_full_parquet`.

1. Load the full data in memory (`open_dataset(...) |> collect()`), and measure the time and the size (`lobstr::obj_size`). Then compute the mean arrival delay and the proportion of flights arriving more than 15 minutes late for each airline, with `dplyr`, with `data.table` and with `duckdb` (through `dbplyr`: register the table in a DuckDB database with `duckdb_register`, then use the same `dplyr` verbs and `collect()`). Check that the results agree, and compare the times and the memory with `bench::mark`. Run the same comparison on the 10% sample: how does each engine scale? Compare also the mean delays estimated on the sample with the exact ones.

2. *Predict before you run:* is it faster to compute on data already in memory, or directly on the Parquet files? Compute the same statistics with Arrow (`open_dataset`, the same `dplyr` verbs and `collect()`) and with DuckDB (an SQL query on `read_parquet('data/flights_full_parquet/*/*.parquet', hive_partitioning = true)`), directly from the files. Check the results, and compare the times and the memory of the R session with the engines of the previous question.

3. Join the results with the full names of the airlines and plot the mean delay and the proportion of late flights. With DuckDB, display the SQL query generated by `dbplyr` (`show_query()`).

Exercise 3 (Profiling and optimizing)

The cumulated arrival delay along the year can be computed with `cumsum`. To understand where the time goes, we write this function ourselves (PC7, *Complexity of cumulative sums*).

1. *Predict before you run:* how does the time of `cumsum_0` grow with the length n of `x`? Measure it for $n = 250, 500, 1000, 2000$ and plot it in log-log scale.

2. Write `cumsum_1` with a single loop (`res[i] <- res[i - 1] + x[i]`) growing the result vector, `cumsum_2` with a preallocated result (`numeric(length(x))`), and compare them with the built-in `cumsum` for $n = 10^5$. Then, if a compiler is available (`pkgbuild::has_build_tools()`), compile the C++ functions of `cumsum.cc` with `Rcpp::sourceCpp` and add them to the comparison.

3. Profile the computation of the cumulated delay along the year (sort the flights by date and departure time, compute the cumulated sum, build the dates, plot) with `profvis`. Which step takes the most time? Is `cumsum` worth optimizing?

Exercise 4 (Parallel cross validation)

We want to predict the arrival delay from the information available at the departure. We compare 5 linear models by an 8-fold cross validation (note: in a formula, `DEPARTURE_DELAY/SCHEDULED_TIME` means *nesting*, not a ratio: the ratio is written `I(DEPARTURE_DELAY / SCHEDULED_TIME)`).

1. Compute the 40 cross validation errors sequentially with `map2_dbl`, and display the mean CV RMSE of each model. Which model would you choose?

2. Start 4 background R processes with `mirai::daemons(4)`, and run the same computation in parallel with `map2_dbl(..., .f = in_parallel(\(formula, fold) ..., cv_rmse = cv_rmse, data = flights_model_data, folds = folds))`: the function sent to the workers must be self-contained, its objects being passed explicitly. Compare the times. Then measure the time of a parallel computation whose tasks are tiny (e.g. the mean of 100 numbers, 40 times). Explain the results with PC7, *Speed-up laws*.

Exercise 5 (Serving a model)

The chosen model has to be made available: to people through a web application, to other programs through a REST API.

1. Fit the fourth model on all the data. What is its size in memory? Which of its components take the space? Reduce it with `butcher::butcher()`; then remove by hand the components that `predict` does not need (the QR decomposition matrix `qr$qr`, the `effects`, the `residuals` and the `model` data frame). Check that its predictions are unchanged, and save it in `flights_model.rds`.

The file `flights_api.R` describes a REST API with `plumber2`: comments `#*` above a function declare the route (`@get /predict`), the expected query parameters and their types, and the format of the answer.

2. Start the API in a background R process (`callr::r_bg`), wait until it answers, and query it with `httr2` for a flight of American Airlines with a departure delay of 30 minutes and a scheduled time of 180 minutes. Compare with `predict`. Then write a `testthat` test of the API that does not need a running server (`plumber2::api("flights_api.R")$test_request(fiery::fake_request(url))`).

   The file `flights_app.R` is a `shiny` application, with a `bslib` layout, using the same model: run it from RStudio (*Run App*) or with `shiny::runApp("flights_app.R")`.

To go further

The following blocks are optional and independent: each of them only uses the objects defined in the core part. Their difficulty is marked by stars: $\star$ direct extension, $\star\star$ new method, $\star\star\star$ longer or research-flavored.

Block	Lecture	Stars
Map/Reduce by hand	Map/Reduce	$\star\star$
Larger than memory	Data size	$\star\star$
The memory hierarchy	Hardware	$\star\star$
How many workers?	Parallelization	$\star$
Spark with sparklyr	Distributed computing	$\star\star$
A reproducible pipeline with targets	Workflows	$\star\star$
In-database scoring with orbital	Serving	$\star\star$
Versioning and deploying a model	MLOps	$\star\star$
The same computations in Python	Tools	$\star$

Exercise 6 (**Map/Reduce by hand** $\star\star$)
In the Map/Reduce model, each partition of the data computes a small summary (*map*) and the summaries are merged (*reduce*). The least squares estimate only needs $\mathbb{X}^\top\mathbb{X}$ and $\mathbb{X}^\top\mathbf{Y}$, which are sums over the partitions (PC7, *Map/Reduce for statistics*).

- For each monthly partition of the Parquet dataset, compute the number of flights, the mean and the sum of squared deviations of the arrival delay, and the matrices $\mathbb{X}_k^\top\mathbb{X}_k$ and $\mathbb{X}_k^\top\mathbf{Y}_k$ for the model `ARRIVAL_DELAY ~ DEPARTURE_DELAY + SCHEDULED_TIME`. Merge them, and compare the mean, the variance and the coefficients with the computations on the whole data.

Exercise 7 (**Larger than memory** $\star\star$)
DuckDB can compute on data larger than the memory it is allowed to use: it streams through the files in chunks, and writes its intermediate results to disk if needed. A median or a quantile is harder than a mean: it is not a mergeable statistic (PC7, *Map/Reduce*), so the values of each group have to be kept or sorted.

- If the full CSV file `data/flights.csv.gz` is available (it is not downloaded automatically: 195 MB), limit the memory of DuckDB to 500 MB (`SET memory_limit = '500MB'`) and compute, directly from the compressed CSV file, the number of flights, the median and the 90% quantile of the arrival delay of the 10 busiest origin airports, with their names (a join with `data/airports.csv`). Measure the time and the memory of the R session.

Exercise 8 (**The memory hierarchy** $\star\star$)
R matrices are stored column by column: reading a column reads contiguous memory, reading a row jumps from one column to the next. The caches of the processor load memory by blocks (*cache lines*), so that contiguous accesses are much faster.

- For a 5000×5000 matrix, compare the time to sum its columns one by one (`m[, j]`) and its rows one by one (`m[i, ]`). Compare also `colSums` and `rowSums`.

Exercise 9 (**How many workers?** $\star$) - Run the 40 tasks of the cross validation of Part 4 with 1, 2, 4 and 8 `mirai` workers (`mirai::daemons(k)`), and plot the speed-up against the number of workers, with the number of cores of your machine (`parallel::detectCores()`). Fit Amdahl's law $S(P) = 1/((1-f) + f/P)$ to the measured speed-ups. What limits the speed-up? (PC7, *Speed-up laws*.)

Exercise 10 (**Spark with sparklyr** $\star\star$)

Spark distributes the data and the computations over a cluster; locally, it runs on the cores of the machine, which is enough to learn its interface. `sparklyr` translates the `dplyr` verbs into Spark SQL, and gives access to the distributed machine learning library MLlib. Spark needs Java (17 or 21) and is installed once with `spark_install()` (about 400 MB). On Linux, macOS and Colab, nothing else is needed. On Windows, Spark also needs Hadoop's `winutils.exe`, a small program that is not shipped with Spark: download it once (from the `winutils` repository of Steve Loughran on GitHub, the source suggested by `sparklyr`) and copy it into the folder `tmp\hadoop\bin` of the Spark installation (given by `sparklyr::spark_home_dir()`), where `sparklyr` looks for it. The next chunk does it (it is not run automatically); without it, `spark_connect` stops with a message giving the same instructions.

- Copy the flights to a local Spark session, compute the mean delay by airline with `dplyr` verbs and with an SQL query, and fit the third model with `ml_linear_regression`. Compare its coefficients with `lm`.

Exercise 11 (**A reproducible pipeline with targets** $\star\star$)

A pipeline declares the steps of an analysis and their dependencies; `targets` only reruns the steps whose code or inputs changed, and can run independent steps in parallel (with `crew`).

- Write a `_targets.R` file with three targets (read the Parquet data, fit the model, compute its RMSE), run it twice with `tar_make()` in a temporary folder, and check that nothing is recomputed the second time. Then change the model and run it again.

Exercise 12 (**In-database scoring with orbital** $\star\star$)

Instead of moving the data to the model, the model can be moved to the data: `orbital` translates a fitted `tidymodels` workflow into SQL expressions, which the database computes.

- Fit the third model as a `tidymodels` workflow, convert it with `orbital::orbital()`, and compute the predictions inside DuckDB (`predict(orbital_object, flights_db)`). Display the generated SQL and check the predictions.

Exercise 13 (**Versioning and deploying a model** $\star\star$)

`vetiver` bundles a model with the metadata needed to serve it, versions it on a *board* (`pins`: a folder, S3, Posit Connect…), and generates the files of an API and of a Docker image.

- Create a `vetiver_model` from the workflow of the previous block, pin two versions of it on a temporary board, list the versions, and write the plumber file and the Dockerfile that would deploy it (`vetiver_write_plumber`, `vetiver_write_docker`). Read the Dockerfile: which parts guarantee that the deployed model is the one you tested?

Exercise 14 (**The same computations in Python** $\star$)

The concepts are the same in Python: `polars` and `duckdb` are fast engines for data frames, `multiprocessing` or `joblib` run parallel tasks, and `FastAPI` serves models.

- Read the Python code below (it is not run here): which R tool corresponds to each step?

```
import polars as pl, duckdb
flights = pl.scan_parquet("data/flights_parquet/**/*.parquet")            # lazy, like open_data
by_airline = flights.group_by("AIRLINE").agg(pl.col("ARRIVAL_DELAY").mean()).collect()
duckdb.sql("SELECT AIRLINE, AVG(ARRIVAL_DELAY) FROM 'data/flights_parquet/**/*.parquet' GROUP BY
from joblib import Parallel, delayed
errors = Parallel(n_jobs=4)(delayed(cv_rmse)(formula, fold) for formula in formulas for fold in
from fastapi import FastAPI
app = FastAPI()
@app.get("/predict")
def predict(departure_delay: float, scheduled_time: float, airline: str): ...
```

Use of AI. Artificial intelligence tools have been used to revise the content of this lab. The main contributor takes full responsibility for this content.

License. The texts of this lab are under the Creative Commons Attribution-ShareAlike 4.0 license (CC BY-SA 4.0), and its code under the MIT license; the datasets keep their own licenses. Main contributor: E. Le Pennec. Contributors: S. Boucheron, A. Dieuleveut, A.K. Fermin, S. Gadat, S. Gaiffas, B. Goujaud, A. Guilloux, Ch. Keribin, Th. Klein, E. Matzner, J. Read, M. Sangnier, E. Scornet.